Lecture L15

*Blind review — please score this lecture on row **L15** in the Excel form (1–10 per dimension).*

---

1. Why We Need Backpropagation

The Problem

You've just designed a neural network with a million parameters. Training requires the gradient of a scalar cost $J(\theta)$ with respect to every one of those parameters. If you try to estimate each partial derivative by perturbing parameters one at a time and measuring the change in cost — the finite-difference method — you'll need at least a million forward passes per gradient evaluation. That's impossibly slow.

You might then think: why not just write down the full derivative symbolically and evaluate it? That gives exact gradients. But here's the trap: the output of a deep network is a deeply nested composition of functions — layer applied to layer applied to layer. When you expand the derivative of such a composition, the same intermediate quantities appear in hundreds of different derivative terms. A naive symbolic approach recomputes them every time they appear. The computational cost grows exponentially with network depth. For a 50-layer network, you'd need more operations than atoms in the universe.

The tension: We need exact gradients. We cannot afford to compute them naively. Is there a way to have both — exactness and efficiency — or are we forced to approximate?

Pause and predict. Before reading the resolution, take 30 seconds to sketch an answer. You know the chain rule from calculus. You know the network is a composition of functions. What would happen if you applied the chain rule from the *input* side forward versus from the *output* side backward? Which order would force you to recompute the same intermediate derivatives over and over? Your prediction here will make the "aha" of backpropagation far more satisfying than if I just tell you the answer.

What Backpropagation Does

Backpropagation resolves this tension completely. It is **not** an approximation — it computes the exact analytical gradient. And it does so efficiently by exploiting something the finite-difference and naive symbolic approaches both miss: the network's layered, compositional structure. The cost J depends on the output; the output depends on the previous layer's activations; those activations depend on the layer before them; and so on, all the way back to the input.

By applying the chain rule in a carefully chosen order — from the output backward toward the input — every intermediate derivative is computed **once** and reused wherever it is needed. The total number of operations is proportional to the number of operations in the forward pass. For a network with n parameters and L layers, backpropagation gives us the full gradient in $\mathcal{O}(n)$ time, not $\mathcal{O}(n^2)$ or worse.

Key insight: Backpropagation is dynamic programming applied to the chain rule. It stores partial results (the gradients at each node) and reuses them, so no subexpression is ever recomputed. If you've seen memoization in algorithms — storing the results of expensive function calls so you don't repeat them — this is the same principle, applied to the graph of derivative dependencies.

Try this yourself: Before reading further, take a moment to think: if a network is just a composition of functions $f_L \circ f_{L-1} \circ ... \circ f_1$, why does reversing the order of differentiation matter? What is it about the backward order that avoids the exponential explosion? The answer will crystallize as we work through the computational graph.

Why this is surprising. If you've only ever used the chain rule on paper for $f(g(h(x)))$, you've probably always differentiated "outside-in": $\frac{df}{dg} \cdot \frac{dg}{dh} \cdot \frac{dh}{dx}$. That *is* the backward order. But when you write code, the natural instinct is to start at the inputs and push derivatives forward. Backpropagation is the algorithmic realization that the order you learned in calculus class — outside-in — is not just a notational convention; it's the key to avoiding exponential work. The same insight appears in automatic differentiation, where "reverse mode" is exactly this backward sweep.

---

2. Prerequisites: Computational Graphs

Representing a Network as a Graph

A feedforward network is a directed acyclic graph. Every node is a variable (a scalar, vector, or tensor). Every directed edge indicates that the node at the tail is an input to the function that produces the node at the head.

Analogy: Think of this graph as a plumbing system. Water (data) flows forward from the source (inputs) through a series of pipes and valves (operations) to the final tank (the cost J). In the backward pass, we're tracing pressure changes backward: if I increase the pressure at the final tank, how much did each upstream valve contribute to that change? The "pressure" is the gradient signal, and it propagates in reverse through the same pipes.

Example graph for a tiny network:
Input x, weight w, bias b.
- $u^{(1)} = x$
- $u^{(2)} = w$
- $u^{(3)} = b$
- $u^{(4)} = u^{(1)} \cdot u^{(2)}$ (multiplication)
- $u^{(5)} = u^{(4)} + u^{(3)}$ (addition)
- $u^{(6)} = \sigma(u^{(5)})$ (sigmoid activation)
- $u^{(7)} = J$, the scalar cost computed from $u^{(6)}$.

Edges: $u^{(1)} \to u^{(4)}$, $u^{(2)} \to u^{(4)}$, $u^{(4)} \to u^{(5)}$, $u^{(3)} \to u^{(5)}$, $u^{(5)} \to u^{(6)}$, $u^{(6)} \to u^{(7)}$.

We want $\frac{\partial J}{\partial u^{(i)}}$ for every node i, especially the leaves x, w, b (the parameters).

The Chain Rule on a Graph

Let the graph have nodes $u^{(1)}, u^{(2)}, \ldots, u^{(n)}$ with $u^{(n)} = J$. For any node $u^{(i)}$, let children$(u^{(i)})$ be the set of nodes that take $u^{(i)}$ as a direct input — that is, the nodes $u^{(j)}$ such that there is an edge $u^{(i)} \to u^{(j)}$.

The chain rule says:

$$\frac{\partial J}{\partial u^{(i)}} = \sum_{u^{(j)} \in \text{children}(u^{(i)})} \frac{\partial J}{\partial u^{(j)}} \cdot \frac{\partial u^{(j)}}{\partial u^{(i)}}$$

Intuition: To know how much J changes when we wiggle $u^{(i)}$, we look at all the nodes that immediately depend on $u^{(i)}$. For each such child $u^{(j)}$, we already know the "upstream" gradient $\frac{\partial J}{\partial u^{(j)}}$ (how much J changes when that child changes). We multiply it by the local derivative $\frac{\partial u^{(j)}}{\partial u^{(i)}}$ (how much the child changes when $u^{(i)}$ changes). Then we sum over all children, because $u^{(i)}$ can influence J through multiple paths.

Critical observation: This formula expresses $\frac{\partial J}{\partial u^{(i)}}$ entirely in terms of the gradients at its children. If we compute gradients starting from the final node $u^{(n)}$ and moving backward, then by the time we reach

node $u^{(i)}$, the gradients at all its children are already computed and waiting. We evaluate each local derivative $\frac{\partial u^{(j)}}{\partial u^{(i)}}$ exactly once, multiply, and accumulate.

Try this yourself: Why must we sum over children? Look at the graph: $u^{(3)}$ feeds into both $u^{(4)}$ and $u^{(5)}$ in the example above. If you only followed one path, you'd miss the contribution from the other. The sum captures all causal pathways from a node to the final cost. This is the graph-structured generalization of the single-variable chain rule.

A concrete mental test. Before moving on, trace the gradient flow for a node with two children in your mind. Suppose $u^{(i)}$ influences $u^{(j)}$ and $u^{(k)}$, and both ultimately influence J. The chain rule sum says $\frac{\partial J}{\partial u^{(i)}} = \frac{\partial J}{\partial u^{(j)}}\frac{\partial u^{(j)}}{\partial u^{(i)}} + \frac{\partial J}{\partial u^{(k)}}\frac{\partial u^{(k)}}{\partial u^{(i)}}$. Why is this a *sum* and not, say, a product or a max? Because a small change $\Delta u^{(i)}$ causes *both* $u^{(j)}$ and $u^{(k)}$ to change, and each of those changes independently nudges J. The total nudge on J is the sum of the two nudges — they accumulate. If you only took one path, you'd be asking "how much does J change if I change $u^{(i)}$ but *only* let that change propagate through $u^{(j)}$ and magically block it from propagating through $u^{(k)}$?" That's not the derivative; that's a counterfactual. The derivative measures the total effect through *all* paths, so we sum.

Worked Example: A Simple Arithmetic Graph

Before we tackle matrices and neural layers, let's internalize the mechanics on a tiny scalar example.

Consider $J(a, b) = (ab) + \sin(ab)$, with $a, b \in \mathbb{R}$.

Step 1: Build the graph.
- $u^{(1)} = a$
- $u^{(2)} = b$
- $u^{(3)} = u^{(1)} \cdot u^{(2)}$
- $u^{(4)} = \sin(u^{(3)})$
- $u^{(5)} = u^{(3)} + u^{(4)} = J$

Step 2: Forward pass (just compute values given concrete a, b).
Suppose $a = 2, b = 3$. Then $u^{(3)} = 6$, $u^{(4)} = \sin(6) \approx -0.279$, $J = 6 - 0.279 = 5.721$.
(We will keep the symbolic form for the gradient computation.)

Step 3: Backward pass.
We initialize: $\frac{\partial J}{\partial u^{(5)}} = 1$ (the gradient of the cost with respect to itself).

Process node $u^{(5)}$ (the addition).
$u^{(5)} = u^{(3)} + u^{(4)}$. Children of $u^{(5)}$: none (it's the output). But $u^{(5)}$ has parents $u^{(3)}$ and $u^{(4)}$. We want the gradients with respect to those parents.

$$\frac{\partial J}{\partial u^{(3)}} = \frac{\partial J}{\partial u^{(5)}} \cdot \frac{\partial u^{(5)}}{\partial u^{(3)}} = 1 \cdot 1 = 1$$

$$\frac{\partial J}{\partial u^{(4)}} = \frac{\partial J}{\partial u^{(5)}} \cdot \frac{\partial u^{(5)}}{\partial u^{(4)}} = 1 \cdot 1 = 1$$

Process node $u^{(4)}$ (the sine).
$u^{(4)} = \sin(u^{(3)})$. Only child is $u^{(5)}$, but we already have $\frac{\partial J}{\partial u^{(4)}} = 1$. We need the local derivative $\frac{\partial u^{(4)}}{\partial u^{(3)}} = \cos(u^{(3)})$. So:

$$\frac{\partial J}{\partial u^{(3)}} \text{ (from this path)} = \frac{\partial J}{\partial u^{(4)}} \cdot \frac{\partial u^{(4)}}{\partial u^{(3)}}$$

Wait — we already have a gradient for $u^{(3)}$ from the addition node. And that's the point: $u^{(3)}$ feeds into *both* $u^{(4)}$ and $u^{(5)}$. The chain rule says we *accumulate*:

$$\frac{\partial J}{\partial u^{(3)}} = \underbrace{1}_{\text{from } u^{(5)}} + \underbrace{\frac{\partial J}{\partial u^{(4)}} \cdot \cos(u^{(3)})}_{\text{from } u^{(4)}}$$

With our concrete values: $\frac{\partial J}{\partial u^{(3)}} = 1 + 1 \cdot \cos(6) \approx 1 + 0.960 = 1.960$.

Pause here. This accumulation is the entire magic of backpropagation. The node $u^{(3)}$ appears in *two* downstream computations. In a naive forward expansion of the chain rule, you'd compute $\frac{\partial J}{\partial u^{(3)}}$ twice — once when tracing the path through the addition, once when tracing the path through the sine. Backpropagation says: "I already visited $u^{(5)}$ and $u^{(4)}$. I have their gradients in hand. Now I just multiply and add." One visit to $u^{(3)}$, one computation of its local derivative contributions. That's the memoization.

Continue the backward pass to the leaves.
We now need $\frac{\partial J}{\partial u^{(1)}}$ and $\frac{\partial J}{\partial u^{(2)}}$ (the gradients for a and b).

Node $u^{(3)} = u^{(1)} \cdot u^{(2)}$. Its local derivatives:

$$\frac{\partial u^{(3)}}{\partial u^{(1)}} = u^{(2)}, \quad \frac{\partial u^{(3)}}{\partial u^{(2)}} = u^{(1)}$$

We already have $\frac{\partial J}{\partial u^{(3)}} = 1.960$. So:

$$\frac{\partial J}{\partial u^{(1)}} = \frac{\partial J}{\partial u^{(3)}} \cdot \frac{\partial u^{(3)}}{\partial u^{(1)}} = 1.960 \cdot u^{(2)} = 1.960 \cdot 3 = 5.880$$

$$\frac{\partial J}{\partial u^{(2)}} = \frac{\partial J}{\partial u^{(3)}} \cdot \frac{\partial u^{(3)}}{\partial u^{(2)}} = 1.960 \cdot u^{(1)} = 1.960 \cdot 2 = 3.920$$

Verify with the closed-form gradient.
$J(a, b) = ab + \sin(ab)$. Then $\frac{\partial J}{\partial a} = b + b\cos(ab) = 3 + 3\cos(6) \approx 3 + 3(0.960) = 5.880$. Matches.
$\frac{\partial J}{\partial b} = a + a\cos(ab) = 2 + 2\cos(6) \approx 3.920$. Matches.

The punchline. We computed the exact gradient with one forward pass and one backward pass. Every intermediate derivative was computed exactly once. The total work was proportional to the number of edges in the graph — the same order as the forward pass. That's the promise of backpropagation, and it scales to graphs with millions of nodes and arbitrary topology, as long as the graph is acyclic.